**Course Title:HIGH PERFORMANCE COMPUTING-ASS_10**

**Name: K. Lasya**

**Batch - 11**

**HALL TICKET:2303A51724**

**Task 1: MPI Hello World**

Step 1: Create a file named hello.c with the following code:

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    // Initialize MPI
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    // Get process rank
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Get total number of processes

    printf("Hello from rank %d of %d\n", rank, size);
    MPI_Finalize();         // Clean up MPI
    return 0;
}
```

**Step 1:**

Create a file named reduction_bad.c

Implement parallel summation using:

#pragma omp critical

inside the loop.

Compile:

gcc -O2 -fopenmp reduction_bad.c -o reduction_bad

Run with:

export OMP_NUM_THREADS=1

export OMP_NUM_THREADS=4

Record execution time.

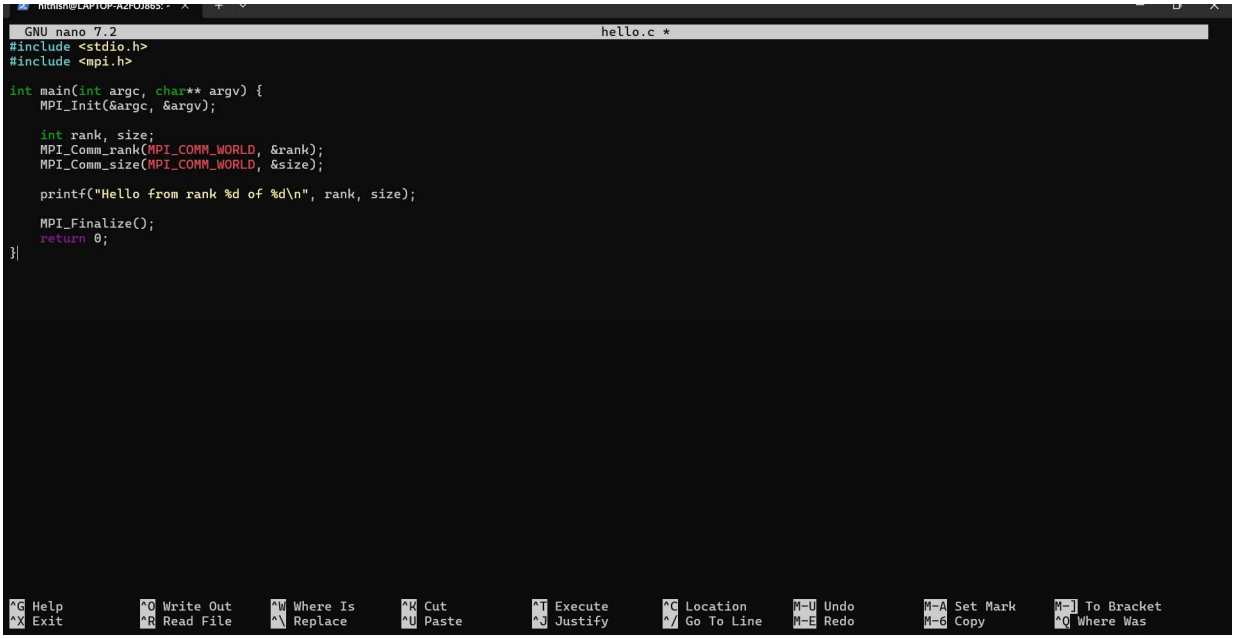
**Step 2: Compile the program using mpicc:**

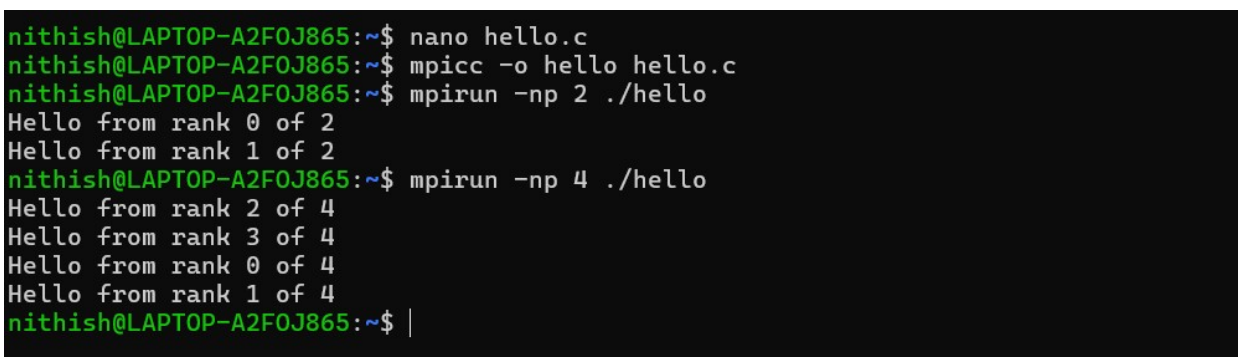
> mpicc -o hello hello.c

**Step 3:** Run the program with different numbers of processes (e.g., 2, 4) and observe the output:

mpirun -np 2 ./hello

npiru -np 4./hello



```
  GNU nano 7.2                                                    reducti
#include <stdio.h>
#include <omp.h>

int main() {

    long long sum = 0;
    int N = 100000000;

    double start = omp_get_wtime();

    for(int i = 0; i < N; i++) {

        sum += i;
    }

    double end = omp_get_wtime();

    printf("Sum = %lld\n", sum);
    printf("Execution Time = %f seconds\n", end - start);

    return 0;
}
```

```
nithish@LAPTOP-A2FOJ865:~$ nano reduction_bad.c
nithish@LAPTOP-A2FOJ865:~$ gcc -O2 -fopenmp reduction_bad.c -o reduction_bad
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=1
./reduction_bad
Sum = 4999999950000000
Execution Time = 0.000000 seconds
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=4
./reduction_bad
Sum = 4999999950000000
Execution Time = 0.000000 seconds
nithish@LAPTOP-A2FOJ865:~$
```

Step 4:Answer the following questions:

1. What do MPI_Init and MPI_Finalize do?
2. What is the range of ranks for a given number of processes?
3. Why can the output order be different each time you run?

1. What do MPI_Init and MPI_Finalize do?

**MPI_Init** is used to start the MPI (Message Passing Interface) environment.
It prepares all the processes so they can communicate with each other during parallel execution. In simple terms, it tells the program: *"Now we are entering parallel mode."*

**MPI_Finalize** is used to safely end the MPI environment.
It cleans up resources and stops communication between processes. After this function, no MPI commands should be used. It basically means: *"Parallel work is finished."*

2. What is the range of ranks for a given number of processes?

Each process in MPI gets a unique **rank (ID number)** so it can be identified.

If there are **N processes**, the ranks start from **0** and go up to **N − 1**.

Example:
If total processes = 4
Then ranks will be: **0, 1, 2, 3**

So, the range of ranks is:

**0 to (number of processes − 1)**

3. Why can the output order be different each time you run?

In MPI programs, multiple processes run **simultaneously** (in parallel).
Since they execute independently and the operating system schedules them differently each time, one process may finish earlier or later in different runs.

Because of this, the print statements from different processes may appear in a **different order** every time you execute the program — even though the program itself is correct.

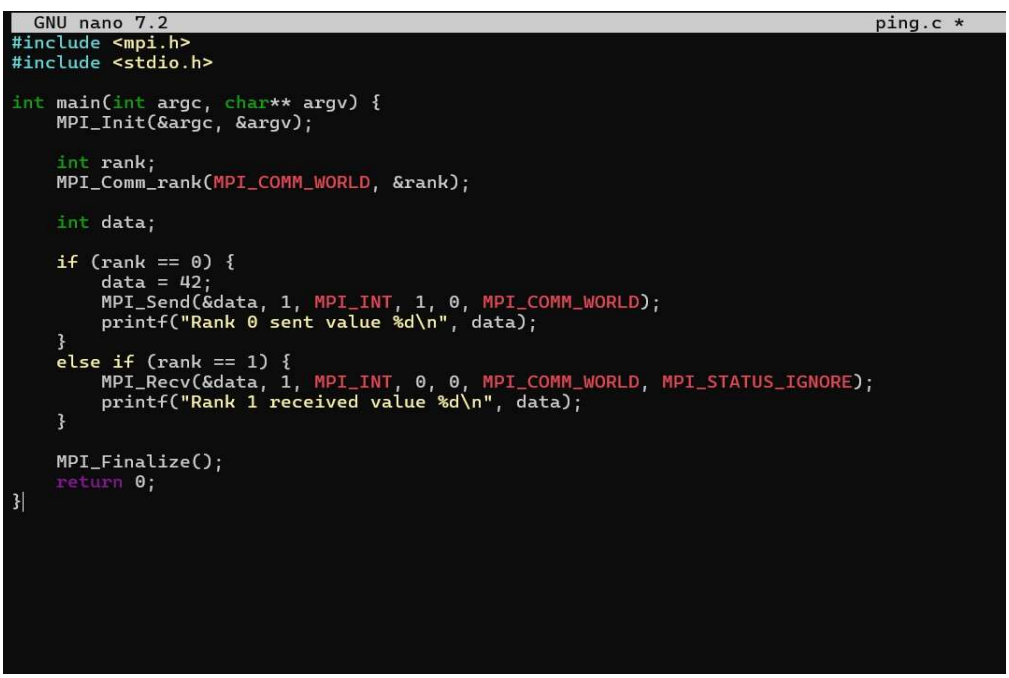
**Task 2: Simple Send/Receive – One-way Communication**

**Step 1:** Create a file named ping.c that sends an integer from rank 0 to rank 1.

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    int data;

    if (rank == 0) {
        data = 42;
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Rank 0 sent value %d\n", data);
    } else if (rank == 1) {
        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Rank 1 received value %d\n", data);
    }

    MPI_Finalize();
    return 0;
}
```



```
nithish@LAPTOP-A2FOJ865:~$ nano ping.c
nithish@LAPTOP-A2FOJ865:~$ mpicc -o ping ping.c
nithish@LAPTOP-A2FOJ865:~$ mpirun -np 2 ./ping
Rank 0 sent value 42
Rank 1 received value 42
nithish@LAPTOP-A2FOJ865:~$
```

**Step 2:** Compile and run with exactly 2 processes:

```
mpicc -o ping pong.c
mpirun -np 2 ./ping
```

```
  GNU nano 7.2                                                    pingpong.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {

    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int data;

    if (rank == 0) {
        data = 42;

        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Rank 0 sent value %d\n", data);

        MPI_Recv(&data, 1, MPI_INT, 1, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Rank 0 received value %d\n", data);
    }
    else if (rank == 1) {

        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Rank 1 received value %d\n", data);

        data = 99;

        MPI_Send(&data, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
        printf("Rank 1 sent value %d\n", data);
    }

    MPI_Finalize();
    return 0;
}
```

```
nithish@LAPTOP-A2FOJ865:~$ mpicc -o pingpong pingpong.c
nithish@LAPTOP-A2FOJ865:~$ mpirun -np 2 ./pingpong
Rank 0 sent value 42
Rank 0 received value 99
Rank 1 received value 42
Rank 1 sent value 99
nithish@LAPTOP-A2FOJ865:~$
```

Step 4: Answer the following:

1. What would happen if you run the program with more than 2 processes? Why?
2. What is the purpose of the tag parameter in MPI_Send and MPI_Recv?
3. What does MPI_STATUS_IGNORE do?

1. What would happen if you run the program with more than 2 processes? Why?

If the program is run with more than 2 processes, only **rank 0 and rank 1** will participate in communication because the code is written only for them.
Other ranks (like 2, 3, etc.) will simply start and finish without doing anything. In some cases, extra processes may remain idle or cause confusion if communication is expected but not defined for them.

2. What is the purpose of the tag parameter in MPI_Send and MPI_Recv?

The **tag** acts like a message label or identifier.
It helps the receiving process know which message it is receiving, especially when multiple messages are being sent between the same processes. This prevents mixing up different messages.

3. What does MPI_STATUS_IGNORE do?

MPI_STATUS_IGNORE tells MPI that the program does not need detailed information about the received message (such as sender details or message size). It simplifies the code when that extra information is not required.

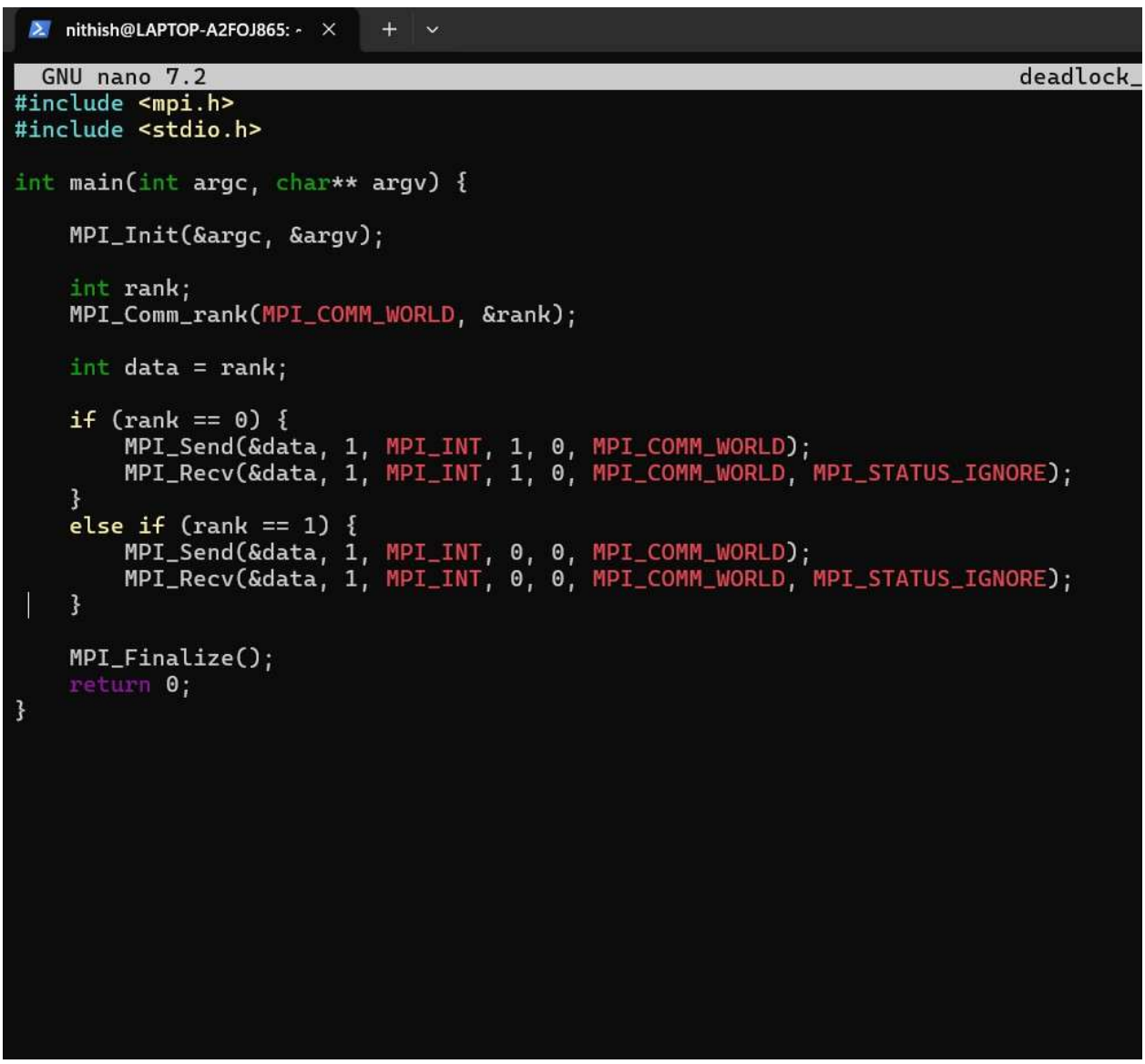
**Task 3: Deadlock and Avoidance**

Step 1: Create a file named deadlock_bad.c that attempts a circular send/receive:

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    int data = rank;

    if (rank == 0) {
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    } else if (rank == 1) {
        MPI_Send(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    }
    MPI_Finalize();
    return 0;
}
```



Step 2: Compile and run with 2 processes. Observe what happens (the program will hang – you may need to kill it with Ctrl+C).

```
nithish@LAPTOP-A2FOJ865:~$ nano deadlock_bad.c
nithish@LAPTOP-A2FOJ865:~$ mpicc -o deadlock_bad deadlock_bad.c
nithish@LAPTOP-A2FOJ865:~$ mpirun -np 2 ./deadlock_bad
nithish@LAPTOP-A2FOJ865:~$ ^C
nithish@LAPTOP-A2FOJ865:~$
```

Step 3: Now create a fixed version deadlock_fixed.c by reordering the calls so that one process receives first:

```
#include <mpi.h>
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    int data = rank;
    if (rank == 0) {
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    } else if (rank == 1) {
        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        MPI_Send(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }
    MPI_Finalize();
    return 0;
```

}

nithish@LAPTOP-A2FOJ865: ~

GNU nano 7.2 deadlock_fix

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {

    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int data = rank;

    if (rank == 0) {
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
    else if (rank == 1) {
        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Send(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

GNU nano 7.2 deadlock_bad.c *

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {

    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int data = rank;

    if (rank == 0) {

        printf("Rank 0: Sending value %d to Rank 1\n", data);
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

        MPI_Recv(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Rank 0: Received value %d from Rank 1\n", data);
    }
    else if (rank == 1) {

        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Rank 1: Received value %d from Rank 0\n", data);

        printf("Rank 1: Sending value %d back to Rank 0\n", data);
        MPI_Send(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```